

IUIU Kampala Campus Academic ERP System

Project folder and file explanation - generated June 15, 2026

This project is a FastAPI-based academic ERP dashboard. The main deployed app starts from `main.py`, which exposes `iuiu_dashboard.api:app`. The app combines an ERP dashboard, NLP translation support, and a course assistant that indexes lecture resources.

High Level Flow

Layer	What It Does
<code>main.py</code>	Imports the integrated FastAPI app from <code>iuiu_dashboard.api</code> . Hosting platforms use this as the entry point.
<code>iuiu_dashboard/api.py</code>	Defines HTTP routes for login, workspaces, resources, quizzes, feedback, PDFs, translation, chat, and admin tools.
<code>iuiu_dashboard/service.py</code>	Contains the main business logic and coordinates the translation and course assistant modules.
<code>data/</code>	Stores local JSON data and uploaded resources. It acts as the project's lightweight database.
<code>static_dashboard/</code>	Provides the main browser interface used by students and lecturers.

Top Level Files

File	What It Does
<code>.env.example</code>	Example environment variables for app port, data paths, translation provider, and course assistant settings.
<code>.gitignore</code>	Prevents Git from tracking virtual environments, caches, local environment files, logs, and local tool settings.
<code>DEPLOYMENT.md</code>	Deployment guide for local runs, Docker, and hosted platforms such as Render, Railway, or Heroku-style services.
<code>Dockerfile</code>	Builds a Python 3.12 container, installs requirements, copies the app, and starts Uvicorn.
<code>IUIU_Kampala_ERP_Final_Project_Report.docx</code>	The final project report document.
<code>main.py</code>	Main deployment entry point. It imports app from <code>iuiu_dashboard.api</code> .
<code>Procfile</code>	Heroku-style process definition for running <code>uvicorn main:app</code> .
<code>README.md</code>	Project overview, feature list, run commands, test command, project structure, and deployment summary.
<code>render.yaml</code>	Render platform configuration: build command, start command, health check path, and environment variables.
<code>requirements.txt</code>	Python package dependencies: FastAPI, Uvicorn, deep-translator, and multipart upload support.
<code>runtime.txt</code>	Python runtime version for compatible hosting platforms.

Main Project Folders

Folder	What It Contains
iuiu_dashboard/	The integrated ERP backend: authentication, student and lecturer workspaces, lecture sessions, quizzes, PDFs, study planning, feedback, and chat.
static_dashboard/	The main browser UI for login, ERP pages, student modules, lecturer modules, and e-learning views.
iuiu_nlp/	The NLP translation module: profiles, nationality language mapping, glossary protection, cache, logs, and providers.
static/	Standalone browser UI for the translation module.
iuiu_course_assistant/	Resource indexing and grounded course-question answering module.
static_course_assistant/	Standalone browser UI for the course assistant.
data/	Local JSON-backed persistence and uploaded lecture resource files.
tests/	Automated unit and API tests for the translation service, course assistant, and integrated dashboard.
nlp_translation_service/	Compatibility wrapper that exposes the NLP app from the older import path.

iuiu_dashboard/

File	What It Does
__init__.py	Exports <code>IntegratedDashboardService</code> for convenient imports.
api.py	FastAPI route layer for the full ERP. It serves the main frontend and exposes authentication, lecturer, student, admin, quiz, feedback, resource, PDF, translation, and chat endpoints.
service.py	Main business logic. Builds demo users, authenticates users, prepares student/lecturer workspaces, creates lecture sessions, generates quizzes, handles quiz attempts, creates study plans and timetables, translates resources, builds PDFs, and coordinates chat.
storage.py	JSON repository classes for dashboard data: timetables, study plans, quizzes, quiz attempts, lecture sessions, and feedback.

static_dashboard/

File	What It Does
index.html	Main ERP HTML shell. Contains the login page and empty containers that JavaScript fills after login.
styles.css	Main stylesheet. Defines the screenshot-style login page, ERP portal chrome, e-learning layout, tables, cards, forms, and responsive behavior.
app.js	Main frontend controller. Handles login/logout, API requests, rendering student and lecturer modules, course rooms, resources, PDFs, quizzes, feedback, chat, timetable forms, study-plan forms, and navigation.
favicon.svg	IUIU browser icon.

iuiu_nlp/

File	What It Does
<code>__init__.py</code>	Package marker and module docstring.
<code>api.py</code>	Standalone translation FastAPI app. Serves the translation UI and routes for health, languages, profiles, glossary, and translation.
<code>config.py</code>	Loads translation settings from environment variables, including data directory, provider, default source language, fallback language, and bilingual behavior.
<code>languages.py</code>	Defines supported languages, maps nationalities to languages, and resolves the final target language from override, profile, nationality, or fallback.
<code>providers.py</code>	Translation provider implementations. Includes demo translation, Deep Translator integration, glossary term protection, and glossary term restoration.
<code>service.py</code>	Translation workflow. Manages profiles, glossary terms, cache lookup, provider translation, bilingual output, and audit logging.
<code>storage.py</code>	JSON repositories for student profiles, glossary, translation cache, audit logs, and nationality-language mappings.

static/ Translation Frontend

File	What It Does
<code>index.html</code>	Standalone NLP translation portal page with profile, glossary, and translation forms.
<code>app.js</code>	Frontend logic for health checks, loading languages, saving profiles, adding glossary terms, sending translation requests, and rendering results.
<code>styles.css</code>	Styling for the standalone translation portal.
<code>favicon.svg</code>	IUIU browser icon.

iuiu_course_assistant/

File	What It Does
<code>__init__.py</code>	Exports CourseAssistantService.
<code>api.py</code>	Standalone course assistant FastAPI app. Serves the course assistant UI and routes for resources, uploads, health, and chat.
<code>config.py</code>	Loads course assistant settings: data directory, upload directory, answer provider, chunk size, overlap, top-k count, and translation toggle.
<code>extractors.py</code>	Extracts text from uploaded files such as text, Markdown, CSV, JSON, HTML, DOCX, and simple PDFs.
<code>providers.py</code>	Answer provider interface and demo grounded answer provider used to build answers from retrieved lecture chunks.
<code>retrieval.py</code>	Tokenizes text, splits notes into chunks, ranks chunks against a question, and selects the best supporting snippet.
<code>service.py</code>	Indexes pasted or uploaded resources, stores uploads, creates searchable chunks, answers course questions with citations, and optionally translates answers.
<code>storage.py</code>	JSON repositories for resources, indexed chunks, and course assistant audit logs.
<code>translation_bridge.py</code>	Connects the course assistant to the NLP translation service when translation is enabled.

static_course_assistant/

File	What It Does
index.html	Standalone course assistant page with resource ingestion, library, and chatbot sections.
app.js	Frontend logic for indexing notes/files, loading resources, asking course questions, and rendering answers, citations, and translations.
styles.css	Styling for the standalone course assistant portal.
favicon.svg	IUIU browser icon.

nlp_translation_service/

File	What It Does
__init__.py	Re-exports app from main.py in this compatibility package.
main.py	Imports and exposes the standalone NLP app from <code>iuiu_nlp.api</code> . This preserves an older deployment/import path.

data/ Root Files

File	What It Stores
glossary.json	Course glossary. Currently includes CSC101 terms such as ERP, quiz, and dashboard with target translations.
nationality_language_map.json	Maps nationalities to default target languages, for example Kenya to Swahili and Uganda to English.
profiles.json	Student translation profiles, nationalities, preferred languages, and update timestamps.
translation_cache.json	Cached translation results so repeated requests can be served without translating again.
translation_logs.json	Audit log of translation requests, cache hits, provider used, glossary terms applied, and timestamps.

data/course_assistant/

File or Folder	What It Stores
resources.json	Metadata for indexed course resources: resource ID, course code, title, topic, version, source type, file name, and timestamps.
resource_chunks.json	Searchable chunks produced from course resources. The course assistant ranks these chunks when answering questions.
course_assistant_logs.json	Course assistant activity log for indexing and chat-answer events.
uploads/	Physical uploaded or generated resource files used by the course assistant.

Files in data/course_assistant/uploads/

File	Meaning
0f241b0c2568-lecture-1.txt, fecf137a690a-lecture-1.txt	CSC101 Lecture 1, ERP foundations.
1ac006b569cf-lecture-2.txt, 8ca45870085e-lecture-2.txt	CSC101 Lecture 2, workflow integration.
358c3b13b3f4-lecture-3.txt, fbb54e1d3638-lecture-3.txt	CSC101 Lecture 3, reports and decision support.
4c4b93e90439-lecture-1.txt, bf2128b411a4-lecture-1.txt	BIT220 Lecture 1, database modelling.
7ccb9a033648-lecture-2.txt, d30a6de1f560-lecture-2.txt	BIT220 Lecture 2, normalization.
ea98fd2dfc60-lecture-3.txt, a090fbd3abbb-lecture-3.txt	BIT220 Lecture 3, query design.
f9c9a301a0d8-lecture-1.txt, 3d7086a9defd-lecture-1.txt	BIS210 Lecture 1, systems overview.
9c63cb72e88b-lecture-2.txt, d7230f336720-lecture-2.txt	BIS210 Lecture 2, requirements and use cases.
e151d441a8b4-lecture-3.txt, ee463615f29d-lecture-3.txt	BIS210 Lecture 3, process modelling.
1651c791a7c6-iuiu-dfds-processmapping-with-gantt-6weeks.docx	Uploaded DOCX project/process mapping document indexed as Week 3 ERP Notes.

data/dashboard/

File	What It Stores
feedback.json	Student feedback submitted to lecturers, including difficulty area, comment, course code, and status.
lecture_sessions.json	Lecture sessions by lecturer/course, including lecture number, title, topic, status, date/week, notes, and attachments.
quizzes.json	Generated quizzes, questions, answer indexes, source resource IDs, and quiz metadata.
quiz_attempts.json	Student quiz submissions, selected answers, grading result, score, percentage, and submission timestamp.
study_plans.json	Generated study recommendations for students, including priority, recommended hours, reasons, and next actions.
timetables.json	Generated student study timetable entries by day, time, course, duration, and focus.

tests/

File	What It Tests
test_service.py	NLP translation service behavior: language resolution, glossary protection, caching, UI route, and compatibility import path.
test_course_assistant.py	Course assistant behavior: indexing, versioning, grounded answers, citations, translation bridge, DOCX extraction, and API routes.
test_dashboard.py	Integrated ERP behavior: authentication, workspaces, lecture sessions, quizzes, PDFs, feedback, study plans, and dashboard API routes.

Local or Generated Folders

Folder	What It Is
.git/	Git history and repository metadata. Do not edit manually.
.venv/	Local Python virtual environment. Generated locally and ignored by Git.
.pytest_cache/	Pytest cache. Generated by test runs.
.mypy_cache/	Type-checking cache. Generated by mypy if used.
.tmp_test/	Temporary test directories. Generated during tests.
__pycache__/	Python bytecode cache. Generated automatically.
.claude/	Local Claude/Codex tool settings. Not part of application logic.

Note: The application currently uses local JSON files as persistence. For deployment, the data paths should point to persistent storage so uploaded resources, profiles, logs, and quiz records survive restarts.